

HW5

Group 12 - Shir, Omer & Ido

19/05/26

Group:

Group members (Name, ID, email)

- Ido Ravid, 207198177, ido.ravid@mail.huji.ac.il
- Omer Sutovsky, 318735107, omer.sutovsky@mail.huji.ac.il
- Shir Tsarfaty, 207718008, shir.tsarfaty@mail.huji.ac.il

Paths and Data

```
if (!require('data.table')) { install.packages('data.table'); library('data.table') }
if (!require('splines'))      { install.packages('splines');   library('splines') }
if (!require('shiny'))        { install.packages('shiny');     library('shiny') }

reads_file <- "../lab2/data/TCGA-13-0723-10B_lib2_all_chr1.forward"
chr1_file  <- "../lab1/data/chr1_str.rda"

BIN_SIZE   <- 5000
beg_region <- 50000000
end_region <- 100000000
```

Reading the Data

```
source("../utils.R")
source("lab5_utils.R")

chr1_reads <- fread(reads_file, col.names = c("Chr", "Loc", "Length"))
load(chr1_file)

# Main analysis region (50-100 Mb, 5000 bp bins)
df_main <- bin_data(chr1_reads, chr1, beg_region, end_region, BIN_SIZE)

cat(sprintf("Loaded %d bins | GC range: %.3f - %.3f | reads range: %d - %d\n",
            nrow(df_main), min(df_main$gc), max(df_main$gc),
            min(df_main$reads), max(df_main$reads)))
```

```
## Loaded 10000 bins | GC range: 0.241 - 0.637 | reads range: 0 - 1258
```

Introduction

Building on labs 3 and 4, lab 5 investigates the GC-coverage relationship in greater depth. We compare the predictive power of G, C, and G+C separately, examine whether the relationship holds consistently across chromosome 1, explore the effect of bin size on correlation strength, and fit Poisson regression models - motivated by the overdispersion we measured previously (VMR 14.7 in lab3, SD_ratio 2.25 in lab4).

Section a — Does C+G explain coverage better than G or C alone?

```
# Four nested models
mod_g <- lm(reads ~ g_prop, data = df_main)
mod_c <- lm(reads ~ c_prop, data = df_main)
#mod_gc <- lm(reads ~ g_prop + c_prop, data = df_main)
#mod_gci <- lm(reads ~ g_prop * c_prop, data = df_main) # includes interaction
#  ↪ g_prop:c_prop
mod_gc_sum <- lm(reads ~ gc, data = df_main)

models_a <- list(mod_g, mod_c, mod_gc_sum)
labels_a <- c("~G", "~C", "~G+C")

# Summary table
coef_table <- do.call(rbind, lapply(seq_along(models_a), function(i) {
  mod <- models_a[[i]]
  cf <- coef(mod)
  ci <- confint(mod)
  r2 <- summary(mod)$r.squared
  # one row per model: just show R² and the non-intercept coefficients as a string
  data.frame(
    Model = labels_a[i],
    R2 = round(r2, 4),
    Coefs = paste(sprintf("%s=%.2f [%s,%s]",
                          names(cf)[-1], cf[-1], ci[-1,1], ci[-1,2]),
                  collapse = " | "),
    stringsAsFactors = FALSE
  )
}))

cat("\n=== Model comparison: R² and coefficients ===\n")
```

```
##
## === Model comparison: R² and coefficients ===
```

```
print(coef_table, row.names = FALSE)
```

```
## Model      R2      Coefs
##   ~G 0.5267 g_prop=1516.42 [1488.24,1544.60]
##   ~C 0.5299 c_prop=1516.26 [1488.26,1544.25]
##   ~G+C 0.6973      gc=1000.75 [987.82,1013.67]
```

```

models_main <- list(
  mod_g,
  mod_c,
  mod_gc_sum
)

labels_main <- c(
  "G alone",
  "C alone",
  "GC = G + C"
)

predictors_main <- list(
  df_main$g_prop,
  df_main$c_prop,
  df_main$gc
)

y <- df_main$reads

baseline_medae <- median(abs(y - median(y)))

model_compare <- do.call(rbind, lapply(seq_along(models_main), function(i) {

  mod <- models_main[[i]]
  y_hat <- predict(mod)
  resid <- y - y_hat

  r2 <- summary(mod)$r.squared

  spearman <- cor(predictors_main[[i]], y, method = "spearman")

  medae <- median(abs(resid))

  #robust_r2 <- 1 - medae / baseline_medae

  data.frame(
    Model = labels_main[i],
    R2 = round(r2, 4),
    Spearman = round(spearman, 4),
    Median_Abs_Error = round(medae, 2)
    #Robust_R2 = round(robust_r2, 4)
  )
}))

print(model_compare)

```

```

##      Model      R2 Spearman Median_Abs_Error
## 1   G alone 0.5267  0.7621          21.18
## 2   C alone 0.5299  0.7551          21.76
## 3 GC = G + C 0.6973  0.8897          14.03

```

The 0.17 difference between the single bases and the shared content is significant and shows that there is a good reason to prefer the GC content over the individual bases.

```

set.seed(42)
keep      <- sample(nrow(df_main), size = min(2000, nrow(df_main)))
df_sub    <- df_main[keep, ]
ord       <- order(df_sub$reads)
reads_sorted <- df_sub$reads[ord]
reads_scaled <- (reads_sorted - min(reads_sorted)) / diff(range(reads_sorted))
pal       <- colorRampPalette(c("#440154", "#21908C", "#FDE725"))(100)
point_cols <- pal[pmax(1, ceiling(reads_scaled * 100))]

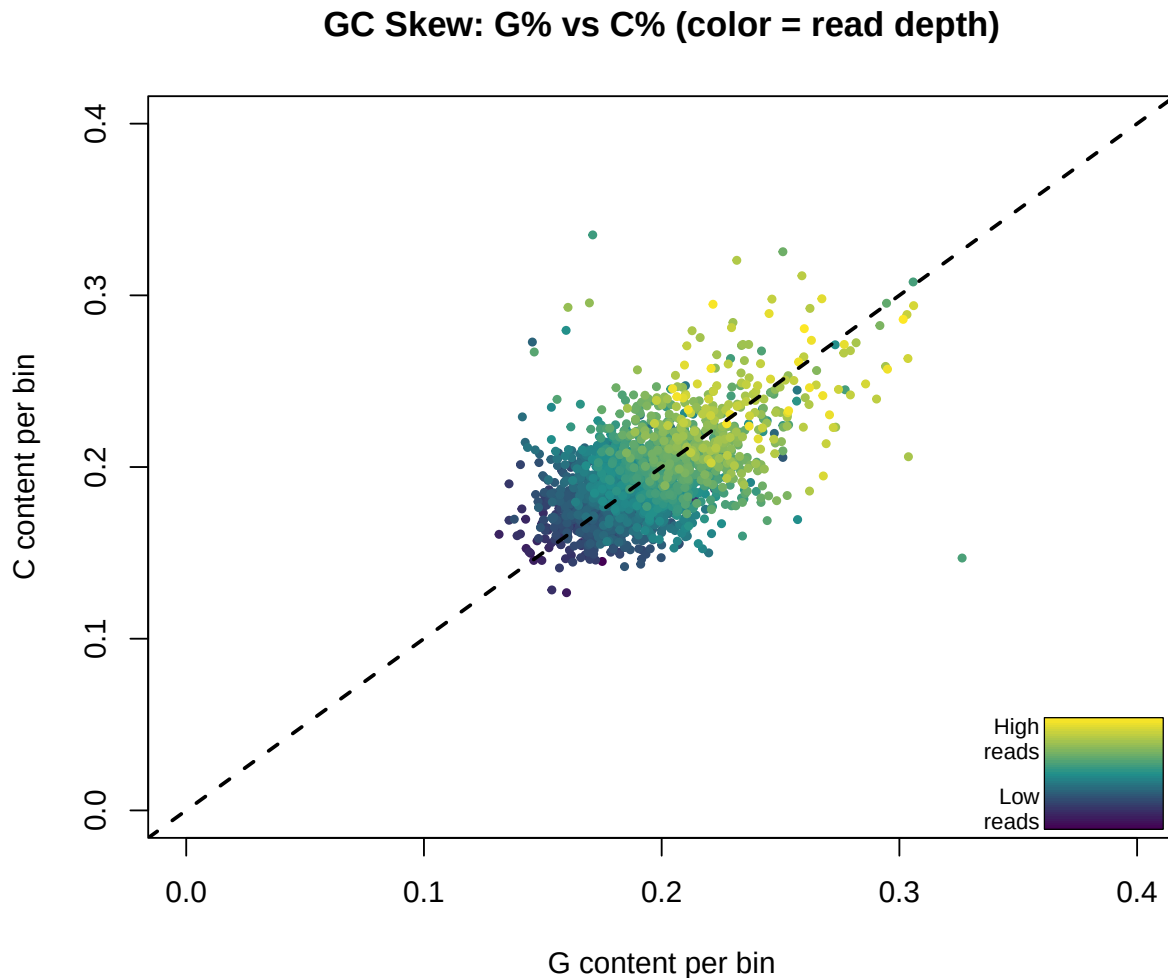
plot(df_sub$g_prop[ord], df_sub$c_prop[ord],
     col = point_cols, pch = 20, cex = 0.8,
     xlab = "G content per bin",
     ylab = "C content per bin",
     main = "GC Skew: G% vs C% (color = read depth)",
     xlim = c(0, 0.4), ylim = c(0, 0.4))

abline(a = 0, b = 1, col = "black", lwd = 2, lty = 2)

mid_x <- mean(par("usr")[1:2])

# Gradient colorbar - bottom right
usr <- par("usr")
bar_x0 <- usr[2] - 0.055; bar_x1 <- usr[2] - 0.005
bar_y0 <- usr[3] + 0.005; bar_y1 <- usr[3] + 0.07
n_seg <- 50
ys <- seq(bar_y0, bar_y1, length.out = n_seg + 1)
bar_pal <- colorRampPalette(c("#440154", "#21908C", "#FDE725"))(n_seg)
for (k in seq_len(n_seg))
  rect(bar_x0, ys[k], bar_x1, ys[k+1], col = bar_pal[k], border = NA)
rect(bar_x0, bar_y0, bar_x1, bar_y1, border = "black", lwd = 0.7)
text(bar_x0 - 0.002, bar_y0, "Low\nreads", adj = c(1, 0), cex = 0.72)
text(bar_x0 - 0.002, bar_y1, "High\nreads", adj = c(1, 1), cex = 0.72)

```



The second graph is our attempt to further explain the difference in explainability. The cloud follows the Chargaff diagonal (G% = C% globally), but notably fans out at higher GC values. Low-GC bins (purple, low reads) sit tightly on the diagonal — G and C are nearly interchangeable there. High-GC bins (yellow, high reads) spread away from it: local G% and C% diverge due to transcription-coupled strand asymmetry in gene-rich regions. This means C carries **independent coverage signal precisely where coverage is highest** — which is why ~G+C outperforms ~G alone despite Chargaff.

Section b — Is the relationship spatially consistent? (Shiny)

The plot below shows spline model R^2 (for model1 from lab4) as a function of genomic position across the full chromosome. Run `shiny::runApp("shiny_app")` for the interactive version with a live slider.

```
chr1_len    <- nchar(chr1)
window_size <- 10000000 # 10 Mb
step_size   <- 5000000  # 5 Mb steps
pos_steps   <- seq(1, chr1_len - window_size, by = step_size)
```

```

r2_profile <- sapply(pos_steps, function(beg_w) {
  df_w <- bin_data(chr1_reads, chr1, beg_w, beg_w + window_size - 1, BIN_SIZE)
  if (nrow(df_w) < 20 || var(df_w$gc) < 1e-10) return(NA)
  knots_w <- quantile(df_w$gc, probs = c(0.25, 0.5, 0.75))
  fit_w <- lm(reads ~ bs(gc, knots = knots_w, degree = 3), data = df_w)
  summary(fit_w)$r.squared
})

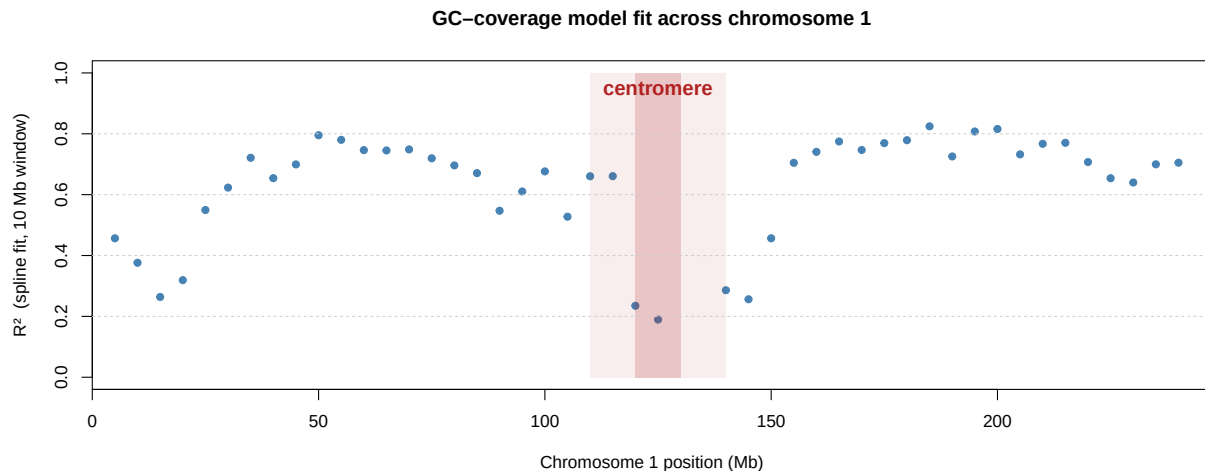
pos_mb <- pos_steps / 1e6

plot(pos_mb + 5, r2_profile,
     type = "p", pch = 19, cex = 0.9, col = "steelblue",
     xlim = c(0, chr1_len / 1e6), ylim = c(0, 1),
     xlab = "Chromosome 1 position (Mb)", ylab = "R2 (spline fit, 10 Mb window)",
     main = "GC-coverage model fit across chromosome 1",
     xaxs = "i")

# Outer band: windows that partially overlap the centromere (~10 Mb buffer each side)
rect(110, 0, 140, 1, col = adjustcolor("firebrick", alpha.f = 0.08), border = NA)
# Core centromere
rect(120, 0, 130, 1, col = adjustcolor("firebrick", alpha.f = 0.20), border = NA)
text(125, 0.95, "centromere", col = "firebrick", cex = 1.2, adj = 0.5, font = 2)

abline(h = seq(0.2, 0.8, by = 0.2), col = "gray80", lty = 2, lwd = 0.7)

```



We can, generally, distinguish between the Centromere and the edges which have poor R^2 scores, and the middle regions between them we classified as “gene-rich”. The gene-rich areas reach around $\sim 0.7 R^2$, while the gene-poor areas perform as worse as $\sim 0.3 R^2$. In the gene-rich areas, the performance is more or less consistent.

Section c — Which bin size gives the strongest relationship?

```

bin_sizes_c <- c(500, 1000, 2000, 5000, 10000, 20000, 50000, 100000, 250000, 500000)

windows_c <- list(
  list(beg = 5000000, end = 25000000, label = "Edge (5-25 Mb)", col =
    ↪ "#377EB8"),
  list(beg = 60000000, end = 80000000, label = "Gene-rich (60-80 Mb)", col =
    ↪ "#4DAF4A"),
  list(beg = 115000000, end = 135000000, label = "Centromere (115-135 Mb)", col =
    ↪ "#E41A1C")
)

r2_all <- matrix(NA, nrow = length(bin_sizes_c), ncol = length(windows_c))
sp_all <- matrix(NA, nrow = length(bin_sizes_c), ncol = length(windows_c))

for (j in seq_along(windows_c)) {
  for (k in seq_along(bin_sizes_c)) {
    df_w <- bin_data(chr1_reads, chr1, windows_c[[j]]$beg, windows_c[[j]]$end,
    ↪ bin_sizes_c[k])
    if (nrow(df_w) < 5 || var(df_w$gc) < 1e-10) next
    mod_w <- lm(reads ~ gc, data = df_w)
    r2_all[k, j] <- summary(mod_w)$r.squared
    sp_all[k, j] <- spearman_r(df_w$gc, df_w$reads)
  }
}

x_labels <- c("500", "1k", "2k", "5k", "10k", "20k", "50k", "100k", "250k", "500k")

# Elbow: first bin where marginal R² gain drops below 5% of total gain (gene-rich region)
r2_gene <- r2_all[, 2]
d1 <- diff(r2_gene)
total_gain <- sum(d1, na.rm = TRUE)
pct_gains <- d1 / total_gain
elbow_idx <- which(pct_gains < 0.05)[1] # index into d1; corresponds to
    ↪ bin_sizes_c[elbow_idx+1]
elbow_x <- bin_sizes_c[elbow_idx + 1]

par(mfrow = c(1, 2), mar = c(5, 4.5, 3, 1))

# R² panel
plot(NULL, xlim = range(bin_sizes_c), ylim = c(0, 1), log = "x",
  xlab = "Bin size (bp)", ylab = "R²",
  main = "R² vs bin size - three genomic regions",
  xaxt = "n")
axis(1, at = bin_sizes_c, labels = x_labels, cex.axis = 0.8)
abline(h = seq(0.2, 0.8, by = 0.2), col = "gray85", lty = 2, lwd = 0.7)
if (!is.na(elbow_x))
  abline(v = elbow_x, col = "gray40", lty = 3, lwd = 1.5)
for (j in seq_along(windows_c))
  lines(bin_sizes_c, r2_all[, j], type = "b", pch = 19, lwd = 2,
    col = windows_c[[j]]$col)
if (!is.na(elbow_x))
  text(elbow_x, 0.08, sprintf("elbow\n%s", x_labels[elbow_idx]),

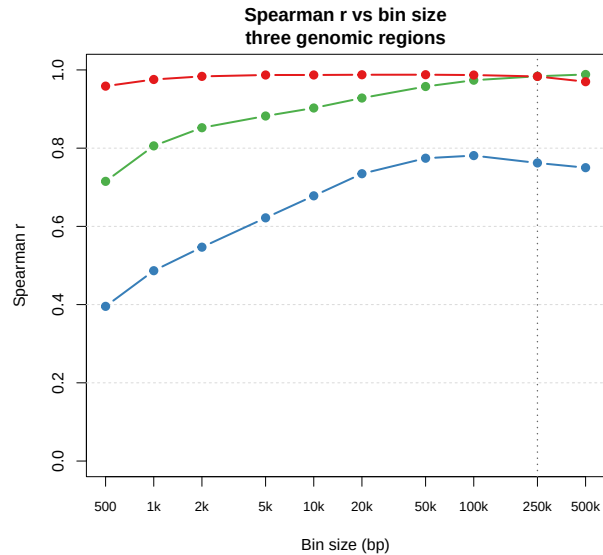
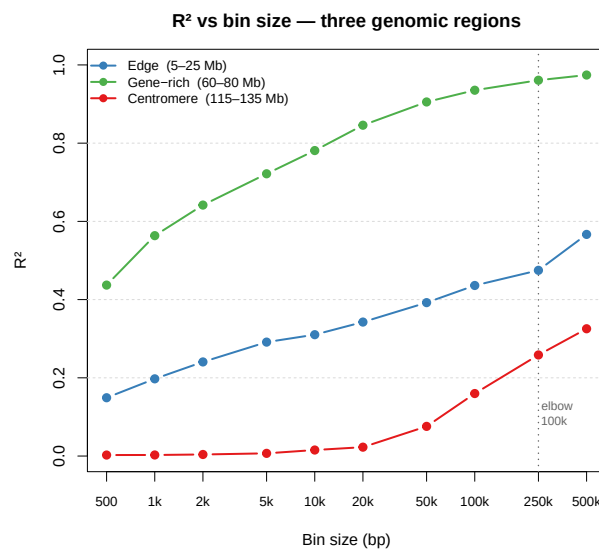
```

```

    col = "gray40", cex = 0.75, adj = c(-0.1, 0))
legend("topleft",
      legend = sapply(window_sizes_c, `[[`, "label"),
      col = sapply(window_sizes_c, `[[`, "col"),
      lwd = 2, pch = 19, bty = "n", cex = 0.85)

# Spearman panel
plot(NULL, xlim = range(bin_sizes_c), ylim = c(0, 1), log = "x",
     xlab = "Bin size (bp)", ylab = "Spearman r",
     main = "Spearman r vs bin size\ntthree genomic regions",
     xaxt = "n")
axis(1, at = bin_sizes_c, labels = x_labels, cex.axis = 0.8)
abline(h = seq(0.2, 0.8, by = 0.2), col = "gray85", lty = 2, lwd = 0.7)
if (!is.na(elbow_x))
  abline(v = elbow_x, col = "gray40", lty = 3, lwd = 1.5)
for (j in seq_along(window_sizes_c))
  lines(bin_sizes_c, sp_all[, j], type = "b", pch = 19, lwd = 2,
        col = window_sizes_c[[j]]$col)

```



```

par(mfrow = c(1, 1))

```

All three regions show the same qualitative trend — R^2 and Spearman increase with bin size as aggregation smooths per-base noise. The curves are vertically separated: the gene-rich region (green) sits consistently highest, the edge region (blue) is intermediate, and the centromere (red) has R^2 near zero regardless of bin size.

The Spearman panel reveals an important nuance: the centromere scores ~ 0.95 Spearman despite $R^2 \approx 0$. This is because Spearman only checks whether rank order is preserved — and in the centromere, the tiny fluctuations that exist happen to be consistent in rank. But coverage is near-zero throughout, so there is no real signal to explain. R^2 correctly reflects this; Spearman is misleading here.

```

bar_cols <- ifelse(pct_gains * 100 < 5, "firebrick", "#4DAF4A")
bar_labels <- x_labels[2:length(x_labels)] # labels for transition points

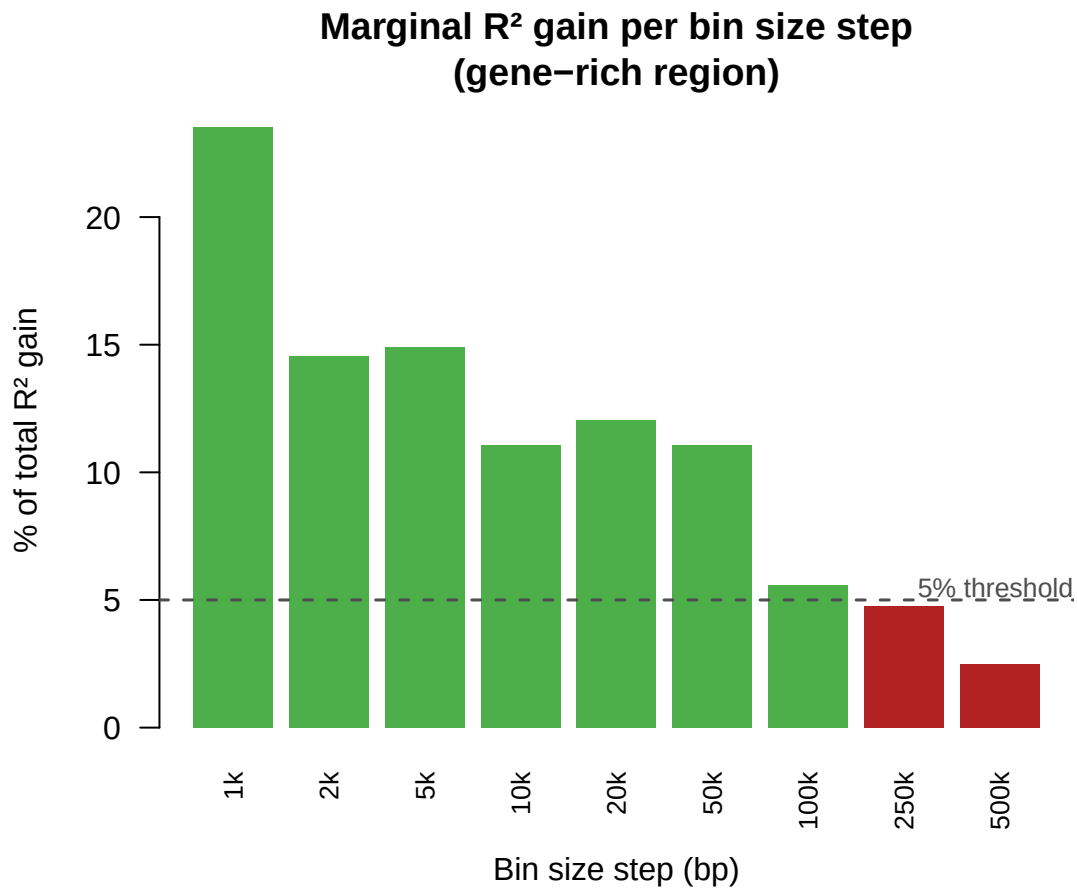
```



```

barplot(pct_gains * 100,
        names.arg = bar_labels,
        col       = bar_cols,
        border    = NA,
        xlab      = "Bin size step (bp)",
        ylab      = "% of total R2 gain",
        main      = "Marginal R2 gain per bin size step\n(gene-rich region)",
        las       = 2,
        cex.names = 0.8)
abline(h = 5, col = "gray30", lty = 2, lwd = 1.5)
text(par("usr")[2], 5.5, "5% threshold", adj = 1, cex = 0.8, col = "gray30")

```



The bar chart shows what fraction of the total R² gain is contributed by each bin size step in the gene-rich region. Green bars are steps that contribute meaningfully (5% of total gain); red bars fall below the threshold and represent diminishing returns. The dashed line at 5% is the stopping criterion — once marginal gains drop below it, increasing bin size costs spatial resolution without meaningfully improving the model.

However, the statistical argument alone is not sufficient. A 100 kb bin may satisfy the threshold, but it would swallow entire genes in a single bin — making it impossible to connect coverage signal back to any specific genomic feature. Human genes are typically 10–100 kb long, and GC-rich regulatory elements such as CpG islands are only 0.5–2 kb. At 5 kb bins, we get enough resolution to detect GC variation *within* genes, distinguish neighboring genes with different GC content, and partially capture CpG island signal — while still having enough reads per bin for stable estimates. The optimal bin size is therefore bracketed

by two constraints: the statistical elbow from above, and the scale of biologically meaningful features from below. 5 kb sits in the middle of that window.

Section d — Poisson Regression

```
# lm baseline
mod_lm <- lm(reads ~ gc, data = df_main)

# Poisson GLM (log link, variance = mean)
mod_pois <- glm(reads ~ gc, data = df_main, family = poisson(link = "log"))

# Quasi-Poisson (variance = phi * mean - estimates dispersion directly)
mod_qpois <- glm(reads ~ gc, data = df_main, family = quasipoisson(link = "log"))

phi_hat <- summary(mod_qpois)$dispersion

# McFadden pseudo-R2 for Poisson
pr2_pois <- pseudo_r2(mod_pois)
pr2_qpois <- pseudo_r2(mod_qpois)

cat(sprintf("\n=== Dispersion parameter (phi) from quasi-Poisson: %.2f ===\n", phi_hat))
```

```
##
## === Dispersion parameter (phi) from quasi-Poisson: 6.95 ===
```

```
cat(sprintf(" (Lab 3 VMR was ~14.7 - consistent with overdispersion of this
↪ magnitude)\n\n"))
```

```
## (Lab 3 VMR was ~14.7 - consistent with overdispersion of this magnitude)
```

```
comparison_d <- data.frame(
  Model = c("lm (Normal)", "glm Poisson", "glm quasi-Poisson"),
  R2_or_pR2 = round(c(summary(mod_lm)$r.squared, pr2_pois, pr2_qpois), 4),
  Metric = c("R2", "McFadden pseudo-R2", "McFadden pseudo-R2"),
  AIC = round(c(AIC(mod_lm), AIC(mod_pois), NA), 1),
  Dispersion = c(NA, 1.0, round(phi_hat, 2)),
  stringsAsFactors = FALSE
)

cat("=== Model comparison table ===\n")
```

```
## === Model comparison table ===
```

```
print(comparison_d, row.names = FALSE)
```

```
##           Model R2_or_pR2           Metric           AIC Dispersion
##      lm (Normal)   0.6973             R2  95662.8           NA
##      glm Poisson   0.4750 McFadden pseudo-R2 137331.4         1.00
##      glm quasi-Poisson      NA McFadden pseudo-R2      NA         6.95
```

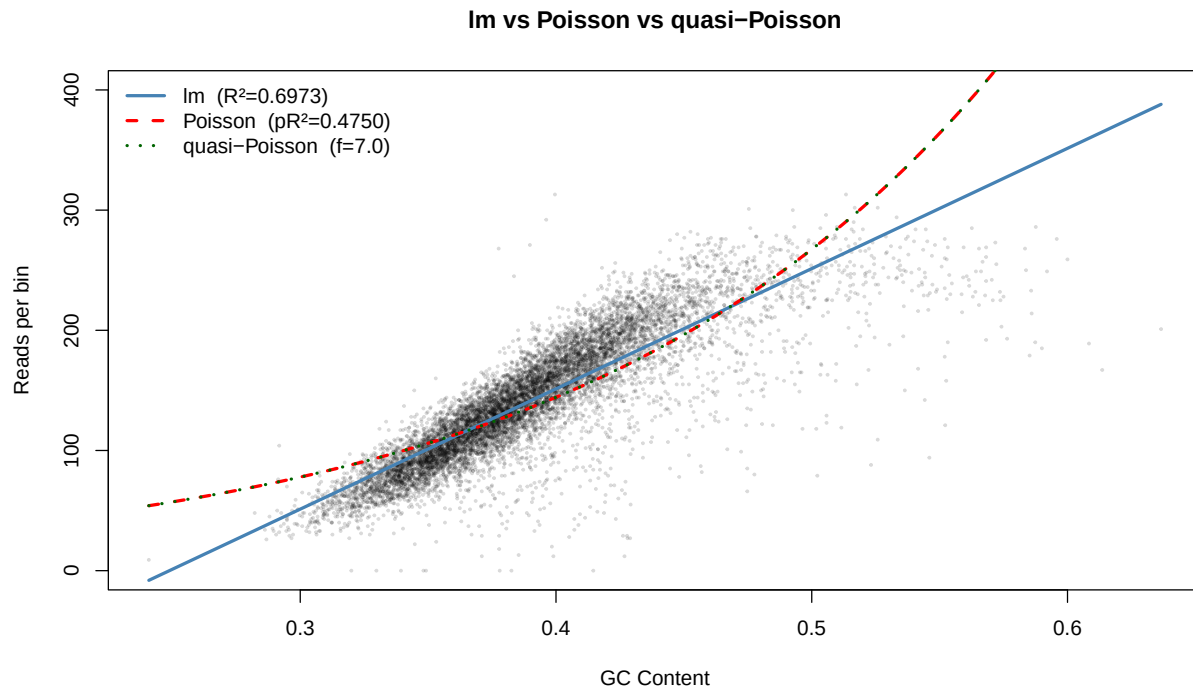
```

gc_seq_d <- seq(min(df_main$gc), max(df_main$gc), length.out = 300)
pred_df_d <- data.frame(gc = gc_seq_d)

preds_lm <- predict(mod_lm, newdata = pred_df_d)
preds_pois <- predict(mod_pois, newdata = pred_df_d, type = "response")
preds_qpois <- predict(mod_qpois, newdata = pred_df_d, type = "response")

plot(df_main$gc, df_main$reads,
     col = rgb(0, 0, 0, 0.15), pch = 20, cex = 0.4,
     xlab = "GC Content", ylab = "Reads per bin",
     main = "lm vs Poisson vs quasi-Poisson",
     ylim = c(0, 400))
lines(gc_seq_d, preds_lm, col = "steelblue", lwd = 2.5)
lines(gc_seq_d, preds_pois, col = "red", lwd = 2.5, lty = 2)
lines(gc_seq_d, preds_qpois, col = "darkgreen", lwd = 2.5, lty = 3)
legend("topleft",
     legend = c(
       sprintf("lm (R²=%.4f)", summary(mod_lm)$r.squared),
       sprintf("Poisson (pR²=%.4f)", pr2_pois),
       sprintf("quasi-Poisson (f=%.1f)", phi_hat)
     ),
     col = c("steelblue", "red", "darkgreen"),
     lwd = 2.5, lty = c(1, 2, 3), bty = "n", cex = 1.0)

```



```

y_max <- max(var_summary$bin_vmr, phi_hat, 1, na.rm = TRUE) * 1.12

plot(var_summary$gc_center, var_summary$bin_vmr,
     type = "b",
     pch = 19,

```

```

lwd = 1.3,
cex = 1.1,
ylim = c(0, y_max),
xlab = "Mean GC content within quantile bin",
ylab = "Empirical variance / fitted mean",
main = "Observed overdispersion across GC bins")

abline(h = 1, col = "red", lwd = 3, lty = 2)
abline(h = phi_hat, col = "darkgreen", lwd = 4, lty = 3)

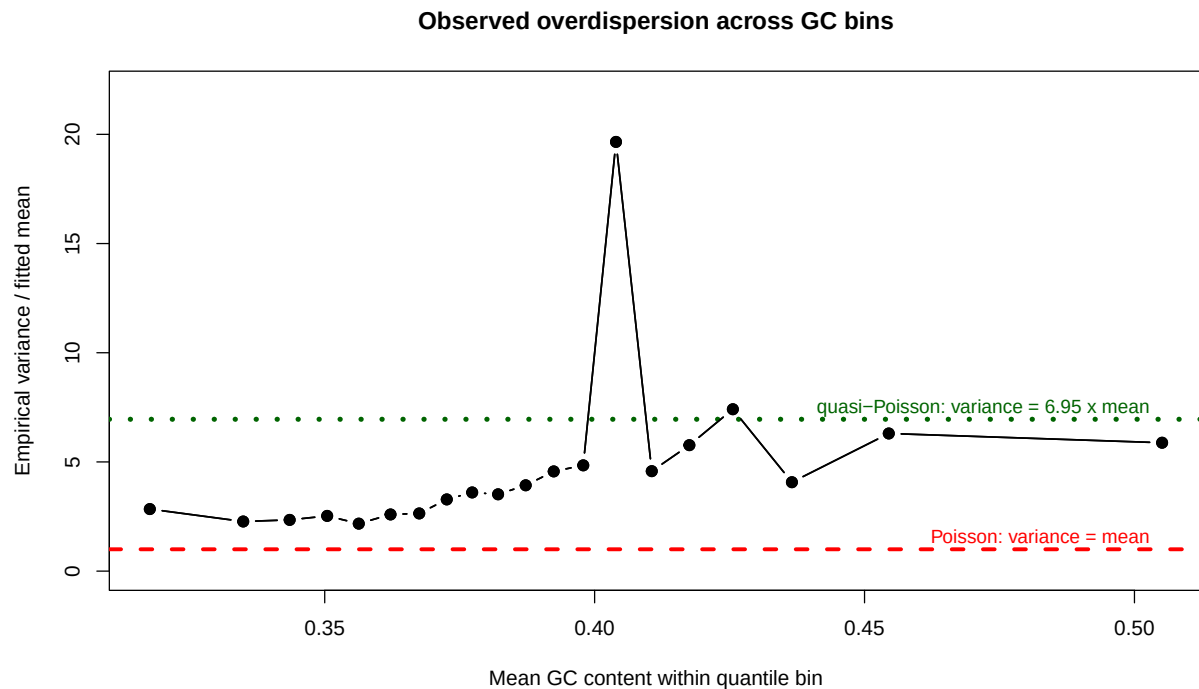
lines(var_summary$gc_center, var_summary$bin_vmr,
      type = "b", pch = 19, lwd = 1.3, cex = 1.1, col = "black")

x_right <- max(var_summary$gc_center)

text(x_right, 1 + 0.5,
     labels = "Poisson: variance = mean",
     col = "red", pos = 2, cex = 0.9)

text(x_right, phi_hat + 0.5,
     labels = sprintf("quasi-Poisson: variance = %.2f x mean", phi_hat),
     col = "darkgreen", pos = 2, cex = 0.9)

```



The linear model and the Poisson models differ in the shape of the fitted mean trend. The linear model fits a straight line on the original reads scale, while the Poisson and quasi-Poisson models use a log link, so their fitted trend is curved on the reads scale. In the central GC range, the fitted curves are fairly similar, but at higher GC values the Poisson curve rises more sharply.

The Poisson and quasi-Poisson fitted curves are almost identical because they use the same log-linear mean structure. Their main difference is not the fitted mean curve, but the variance assumption. Ordinary Poisson

regression assumes that the variance equals the mean, while quasi-Poisson allows the variance to be 7 times the mean. Here, the estimated dispersion is 7, meaning that after accounting for GC, the remaining variance is about 7 times larger than expected under an ordinary Poisson model.

This should not be interpreted as exactly the same quantity as the raw VMR from lab 3. The VMR measured the total marginal overdispersion before modeling GC, while the quasi-Poisson measures the remaining overdispersion after GC is included as a predictor. Therefore, the lower value suggests that GC explains part of the extra variation, but substantial overdispersion remains.

Short Summary

We examined the GC-coverage relationship from four angles.

In (a), we found that G and C are nearly interchangeable predictors (Chargaff symmetry), and the interaction term adds negligible explanatory power.

In (b), the spatial analysis showed that the strength of the GC-coverage relationship varies considerably across chromosome 1, with R^2 ranging from near-zero in low-complexity regions (centromere, telomere) to above 0.8 in gene-rich regions.

In (c), larger bin sizes generally increased R^2 , because aggregation smooths out per-base noise — though with diminishing returns beyond ~5000 bp.

In (d), Poisson and quasi-Poisson regression confirmed what we suspected since lab 3: the data are overdispersed (≈ 14), and a quasi-Poisson model provides a more honest fit than either plain Poisson or linear regression.

Section e — AI Usage in This Lab

This lab was explicitly AI-first. We used Claude Code throughout: to design the analysis plan, write the utility function library (`lab5_utils.R`), scaffold the Shiny app, debug graphical parameter issues, and refine the narrative. Key decisions — model selection, which comparisons to make, what to highlight in the presentation — were made by us; Claude handled implementation speed. The Shiny interactive explorer was designed collaboratively: we described the desired interaction (slider + live plot update) and Claude implemented it.

External Sources Statement

We used AI (Claude Code) to:

1. Design the overall analysis plan for this lab
2. Write and debug `lab5_utils.R` utility functions
3. Build the Shiny interactive application
4. Fix plot layout and legend positioning
5. Connect quasi-Poisson dispersion back to lab 3/4 findings